P2845R0
Formatting of std::filesystem::path

Published Proposal, 2023-05-07

Author:
Victor Zverovich

Audience:
SG16

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Introduction

2 Problems

3 Proposal

4 Wording

5 Implementation

References

Informative References

"The Tao is constantly moving, the path is always changing." — Lao Tzu

§ 1. Introduction

[P1636] "Formatters for library types" proposed adding a number of `std::formatter` specializations, including the one for `std::filesystem::path`. However, SG16 recommended removing it because of quoting and localization concerns. The current paper addresses these concerns and proposes adding an improved `std::formatter` specialization for `path`.

2. Problems

[P1636] proposed defining a `formatter` specialization for `path` in terms of the `ostream` insertion operator which, in turn, formats the native representation wrapped in `quoted`. For example:

```
std::cout << std::format("{}, std::filesystem::path("/usr/bin"));
```

would print `"/usr/bin"` with quotes being part of the output.

Unfortunately this has a number of problems, some of them raised in the LWG discussion of the paper.

First, `std::quoted` only escapes the delimiter (`"`) and the escape character itself (`\`). As a result the output may not be usable if the path contains control characters such as newlines. For example:

```
std::cout << std::format("{}", std::filesystem::path("multi\nline"));
```

would print

```
"multi
line"
```

which is not a valid string in C++ and many other languages, most importantly including shell languages. Such output is pretty much unusable and interferes with formatting of ranges of paths.

Another problem is encoding. The `native` member function returns `basic_string<value_type>` where

> `value_type` is a `typedef` for the operating system dependent encoded character type used to represent pathnames.

`value_type` is normally `char` on POSIX and `wchar_t` on Windows.

This function may perform encoding conversion per [fs.path.type.cvt].

On POSIX, when the target code unit type is `char` no conversion is normally performed:

> For POSIX-based operating systems `path::value_type` is `char` so no conversion from `char` value type arguments or to `char` value type return values is performed.

This usually gives the desired result.

On Windows, when the target code unit type is `char` the encoding conversion would result in invalid output. For example, trying to print the following path in Belarusian

```
std::print("{}\n", std::filesystem::path(L"Шчучыншчына"));
```

would result in the following output in the Windows console even though all code pages and localization settings are set to Belarusian and both the source and literal encodings are UTF-8:

```
"???????????"
```

The problem is that despite `print` and `path` both support Unicode the intermediate conversion goes through CP1251 (the code page used for Belarusian) which is not even valid for printing in the console which uses legacy CP866. This has been discussed at length in [P2093] "Formatted output".

§ 3. Proposal

Both of the problems discussed in the previoius section have already been solved. The escaping mechanism that can handle invalid code units has been introduced in [P2286] "Formatting Ranges" and encoding issues have been addressed in [P2093] and other papers. We apply those solutions to the formatting of paths.

This paper proposes adding a `formatter` specialization for `path` that does escaping similarly to [P2286] and Unicode transcoding on Windows.

Code	Before	After
`auto p = std::filesystem::path("multi\nline");` `std::cout << std::format("{}", p);`	`"multi` `line"`	`"multi\nline"`

`// On Windows with UTF-8 as a literal encoding.`
`auto p = std::filesystem::path(L"Шчучыншчына");`
`std::print("{}\n", p);`
	`"???????????"`	`"Шчучыншчына"`

This leaves only one question of how to handle invalid Unicode. Plain strings handle them by formatting ill-formed code units as hexadecimal escapes, e.g.

```
// invalid UTF-8, s has value: ["\x{c3}("]
std::string s = std::format("[{:?}]", "\xc3\x28");
```

This is useful because it doesn't loose any information. But in case of paths it is a bit more complicated because the string is in a different form and the mapping between ill-formed code units in one form to another may not be well-defined.

The current paper proposes applying hexadecimal escapes to the original ill-formed data because it gives more intuitive result and doesn't require non-standard mappings such as WTF-8 ([WTF]).

For example:

```
auto p = std::filesystem::path(L"\xd800"); // a lone surrogate
std::print("{}\n", p);
```

prints

```
"\u{d800}"
```

§ 4. Wording

Add to "Header <filesystem> synopsis" [fs.filesystem.syn]:

```
// [fs.path.fmt], formatter
template<class charT> struct formatter<filesystem::path, charT>;
```

Add a new section "Formatting" [fs.path.fmt] under "Class path" [fs.class.path]:

```
template<class charT> struct formatter<filesystem::path, charT> {
  constexpr format_parse_context::iterator parse(format_parse_context& ctx);

  template<class FormatContext>
    typename FormatContext::iterator
      format(const filesystem::path& path, FormatContext& ctx) const;
};
```

```
constexpr format_parse_context::iterator parse(format_parse_context& ctx);
```

Effects: Parses the format specifier as a *path-format-spec* and stores the parsed specifiers in `*this`.

path-format-spec:
fill-and-align$_{opt}$ *width*$_{opt}$

Returns: An iterator past the end of the *path-format-spec*.

```
template<class FormatContext>
  typename FormatContext::iterator
    format(const filesystem::path& p, FormatContext& ctx) const;
```

Effects: Writes escaped ([format.string.escaped]) `p.native()` into `ctx.out()`, adjusted according to the *range-format-spec*.

Returns: An iterator past the end of the output range.

§ 5. Implementation

The proposed `formatter` for `std::filesystem::path` has been implemented in {fmt} ([FMT]).

§ References

§ Informative References

[FMT]
Victor Zverovich; et al. The fmt library. URL: https://github.com/fmtlib/fmt

[P1636]
Lars Gullik Bjønnes. Formatters for library types. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1636r2.pdf

[P2093]
Victor Zverovich. Formatted output. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2093r14.html

[P2286]
Barry Revzin. Formatting Ranges. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2286r8.html

[WTF]
Simon Sapin. The WTF-8 encoding. URL: https://simonsapin.github.io/wtf-8/